

Workshop No. 2

Data System Architecture and Information Retrieval

Luis Sebastian Martinez Guerrero & Leidy Marcela Morales Segura *

Univesidad Distrital Francisco Jose de Caldas

May 29, 2025

Abstract

In the competitive landscape of e-commerce, platforms like Mercado Libre demand robust, scalable, and secure database systems to manage millions of daily transactions, diverse user interactions, and real-time analytics. This workshop solution focuses on designing a high level database architecture tailored to such needs, integrating strategic business alignment, user-centric workflows, and cutting-edge technical infrastructure.

1 Introduction

See the workshop website for more instructions:

<https://github.com/SebasMa24/Course-DataBases2>

In the competitive landscape of e-commerce, platforms like Mercado Libre demand robust, scalable, and secure database systems to manage millions of daily transactions, diverse user interactions, and real-time analytics. This workshop solution focuses on designing a high level database architecture tailored to such needs, integrating strategic business alignment, user-centric workflows, and cutting-edge technical infrastructure.

In a highly competitive digital environment like e-commerce, platforms like Mercado Libre face the challenge of processing and analyzing large volumes of information generated by millions of users, products, interactions, and transactions in real time. This complexity involves managing both critical transactional operations (such as purchases, payments, and registrations) and unstructured data (such as searches, reviews, or questions). Data fragmentation, query latency, and the difficulty of extracting strategic insights in real time represent barriers to operational efficiency and data-driven decision-making.

Faced with this reality, it is necessary to implement a Database Management System (DBMS) specifically designed to meet the demands of speed, scalability, analysis, and continuous availability in a Big Data context. This system must not only store structured and unstructured information but also ensure that data can be queried, analyzed, and replicated across multiple regions with low response times and high resilience.

2 Definition of the Database Management System

The proposed database management system is conceived as an organized set of interdependent elements, the purpose of which is to store, transform, query, and deliver information relevant to the operation and analysis of an e-commerce platform. This system is designed to meet the demands inherent in a Big Data environment, such as processing large volumes of information in real time, supporting strategic decisions, and ensuring high availability for geographically distributed users.

*Use footnote for providing further information about author (webpage, alternative address)—*not* for acknowledging funding agencies. Funding acknowledgements go at the end of the paper.

2.1 Set of elements

The system's elements are grouped into four key dimensions: people, data, processes, and technological resources, each fulfilling a specific function within the system and essential for its proper functioning.

1. People (system users)

Actors who interact directly or indirectly with the system:

- **Technical users:** database administrators, developers, ETL operators.
- **Functional users:** analysts, sales managers, product supervisors.

2. Data:

Data is the primary input and the backbone of all interrelationships. It includes:

- **Structured data:** users, products, orders, payments, inventory.
- **Unstructured/semi-structured data:** reviews, searches, interactions, questions and answers.
- **Simulated data:** CSV and JSON files generated to represent the behavior of a real-world environment.

3. Processes:

Set of automated activities that transform data into useful information:

- **Data ingestion:** periodic reading of synthetic files.
- **Transformation:** cleansing, normalization, enrichment.
- **Loading:** insertion into operational and analytical databases.
- **Queries:** transactional or analytical operations.
- **Aggregation and recommendation generation:** use of logic to extract value from data.

4. Backend Functional Modules:

System components that organize business logic:

- **User management**
- **Product management**
- **Order and payment management**
- **Review and generated content management**
- **Personalized recommendations module**

5. Databases

Logical infrastructure where information is stored according to its nature:

- **Relational database (transactional):** stores operational data with integrity and consistency.
- **Document database (analytical):** contains flexible, unstructured data used in analysis and recommendations.
- **Caching system:** accelerates access to high-demand data.

6. API Gateway

Entry point for simulated external requests, connecting the system to its environment:

- **RESTful services**
- **Authentication and validation mechanisms**
- **Routing to backend modules**

7. Analytics Tools (BI)

Query and visualization module for functional users:

- **Interactive dashboards**
- **Key performance indicators (KPIs)**
- **Automated or on-demand reports**

2.2 Interrelationship between components

In a complex system like the one proposed for an e-commerce platform, the elements do not operate in isolation, but are strongly interconnected. This functional interdependence ensures that data flows coherently and efficiently from its point of origin to its final use, whether operational or analytical. Each component fulfills a specific role, but its effectiveness depends on proper functioning and synchronization with the other elements.

- **Coordination between processes and data**
One of the fundamental links in the system is the relationship between processes and data. Synthetic data enters the system as flat files or simulated information streams. This data, on its own, is worthless until it is transformed through processes defined in the ETL pipeline. The interaction between these processes and data sources is critical: an error in the transformation process can alter the expected structure, compromising the integrity of the relational model or generating inconsistencies in the analytical repositories. Thus, the system depends on a precise and controlled processing chain, where each stage feeds into the previous one and prepares the next.
- **Communication between backend modules**
The modules that comprise the backend (users, products, orders, payments, reviews) share common entities and constantly communicate with each other. For example, the orders module depends on the products module to verify availability, and on the users module to associate the purchase with a customer. This cross-relationship implies a need for referential integrity and real-time data consistency. Therefore, the transactional database acts as a hub that connects all these modules through well-defined relationships. A change in one module can have an immediate effect on the others, which requires a robust design of validations and business rules.
- **Synchronization between transactional and analytical storage** Another key interrelationship exists between operational and analytical storage systems.
While the relational database stores the structured information needed for real-time operations, the document repository allows for the exploration of behavioral data and informal interactions. The ETL system acts as a bridge between both worlds, ensuring that operational data is extracted, transformed, and loaded consistently into the analytical system. This connection is vital to avoid inconsistencies in dashboards or misalignment between what happens in the operation and what is analyzed in the metrics.
- **Relationship between BI and functional users**
This interrelationship is also evident in the sharing of information. Functional users such as analysts, managers, or marketing managers do not interact directly with the databases, but rather through the business intelligence (BI) module. This tool consumes information from both the transactional and analytical environments, generating visualizations, indicators, and reports. The quality of the final visualization depends entirely on the quality and timeliness of the data entered and transformed by the previous modules, thus closing the cycle of interdependence.
- **External Environment and Interoperability**
Finally, the system is not isolated: it interacts with a simulated external environment, composed of REST clients, data sources, and test applications. These external inputs interact with the system through the API, whose function is to mediate between external requests and internal logic. Another essential interrelationship arises here: the API not only exposes services but also depends on the backend's response and the status of the databases to deliver accurate and timely information. If any of the subsystems fails, the external interface will reflect that problem, demonstrating the systemic and connected nature of the design.

2.3 Objectives

The system's primary purpose is to ensure efficient, reliable, and timely access to data, both for transactional operations and business analysis. This objective responds to the need for the e-commerce platform to make real-time decisions, offer personalized recommendations to users, and maintain data consistency and availability globally.

2.4 Inputs, processes and outputs

Regarding system inputs, the system ingests files in structured and semi-structured formats (CSV and JSON), which contain synthetic data that simulates user behavior, products, orders, and advertising campaigns. In addition, the system receives external requests through programming interfaces (REST APIs), which represent interactions with customers or automated applications. Finally, internally generated data is included, such as browsing logs, user searches, comments, and ratings.

Data processing is performed through an automated workflow that includes validation, transformation, and loading (ETL). This process organizes the information so it can be distributed across two types of storage: one oriented toward transactional operations and the other toward behavioral analysis. During this stage, indexing, partitioning, and caching strategies are implemented to optimize query performance. Derived information such as key performance indicators (KPIs), search-to-purchase conversions, and review classification are also calculated. Part of the processing is intended to feed recommendation modules that contribute to the personalization of the user experience.

The system's outputs consist of pre-structured data, ready to be used by different analysis areas. These include sales reports, consumer behavior, product trends, and campaign results. This information is presented through dashboards that allow strategic stakeholders—such as analysts, marketing managers, and operational managers—to make evidence-based decisions. Additionally, the system provides near-real-time responses to API requests, including queries on product availability, personalized recommendations, and key metrics.

2.5 Critical System Points

3 Model canvas

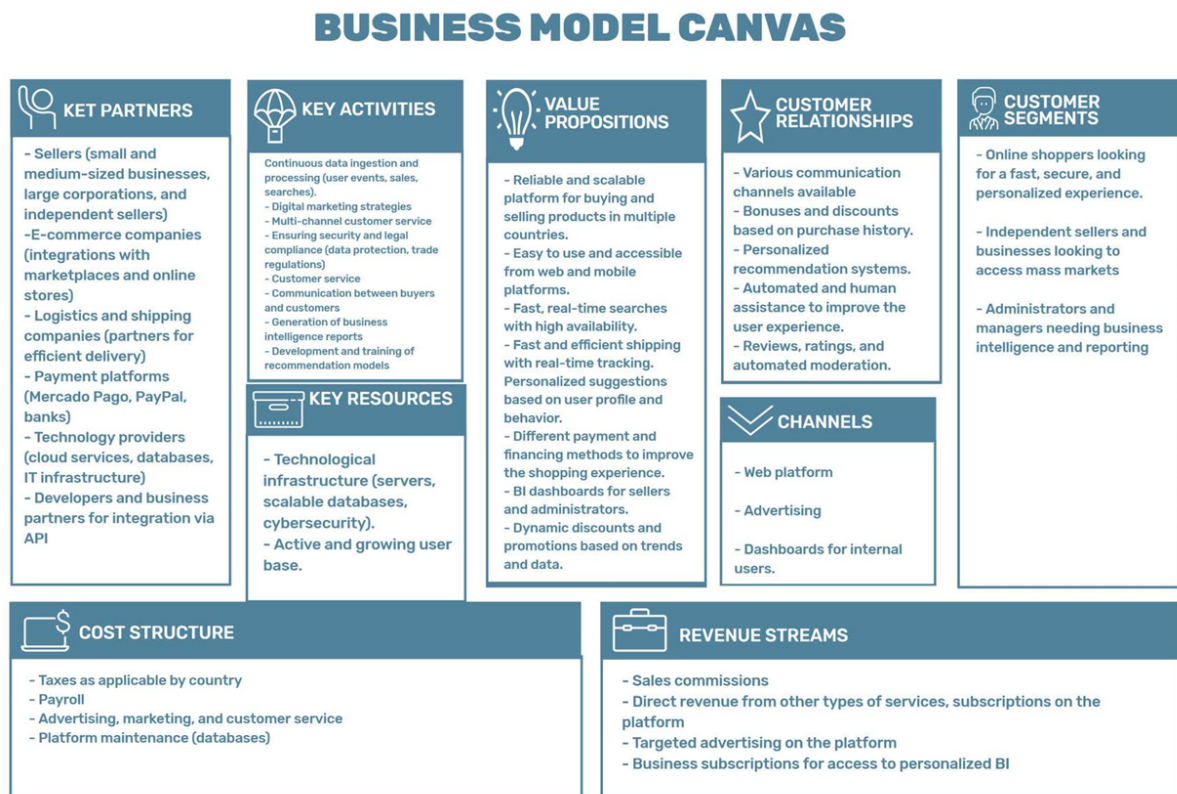


Figure 1: Business model canvas

4 User stories

User stories are changed to a card format for easier viewing.

4.1 Role: User

Title: Buyer Email Registration Priority: High
User story: As a Buyer I want register with my email so what i can access the platform easily
Acceptance Criteria: Given: A user wants to register on the platform. When: The user enters their email address and other required registration data, and the system attempts to validate the email. Then: The system allows registration using the email address. Email format validation is required. An account confirmation email is sent for account activation.

Title: Product Search with Filters Priority: High
User story: As a Buyer I want search for products with filters so what I can find what I need quickly
Acceptance Criteria: Given: I am on the product search page. When: I apply filters by category, price range, or rating. Then: Search results update to show only products that meet the selected filters. Search response time is less than 1 second. The search event is logged in the system.

Title: Personalized Product Recommendations Priority: Medium
User story: As a Buyer I want to see personalized recommendations So that I discover relevant products
Acceptance Criteria: Given: I am Browse the platform or logged into my account. When: The system shows me product recommendations. Then:I see at least 5 suggested products that are relevant to me. Recommendations update dynamically based on my Browse and purchase history.

Title: Pay with Different Methods Priority: High
User story: As a Buyer I want to pay with different methods So that the purchase is more convenient s
Acceptance Criteria: Given: I have reached the payment process for my purchase. When: I select a payment method. Then: I can choose to pay by card, bank transfer, or cash on delivery. Payment confirmation is automatic once the transaction is complete. Real-time error handling is provided, and I am notified of any payment issues.

Title: Leave Reviews for Purchased Products Priority: Medium
User story: As a Buyer I want to leave reviews for purchased products So that I can help other buyers
Acceptance Criteria: Given: I have purchased a product and it has been delivered. When: I attempt to leave a review for that product. Then: I can only leave reviews for products I have previously purchased. I can rate the product with a minimum of 1 star and a maximum of 5 stars. The review is automatically moderated before being published.

4.2 Role: Seller

Title: Create Detailed Product Listings Priority: High
User story: As a Seller I want to create detailed product listings So that I attract more buyers
Acceptance Criteria: Given: I am creating a new product listing. When: I enter the product information. Then: I can complete minimum fields like title, description, price, and stock. I can upload up to 5 images per product.

Title: Monitor Product Statistics Priority: Medium
User story: As a Seller I want to monitor my product statistics So that I can optimize their performance
Acceptance Criteria: Given: I access my seller dashboard. When: I review my product statistics. Then: I see metrics like views, sales, reviews, and stock. I can filter statistics by date and category. Statistics are visualized on a dashboard.

Title: Receive Low-Stock Notifications Priority: High
User story: As a Seller I want to receive notifications about low-stock products So that I can restock them
Acceptance Criteria: Given: I have products in my inventory. When: A product's stock falls below a configured threshold. Then: I receive a notification via email and on the platform. The low-stock threshold is configurable per product.

Title: Apply Temporary Promotions Priority: Medium
User story: As a Seller I want to apply temporary promotions So that I can boost sales
Acceptance Criteria: Given: I have active products in my store. When: I configure a promotion. Then: I can select products in batch for the promotion. I can define a start and end date for the promotion. I can visualize the sales impact of the promotion.

Title: Manage User Questions Priority: High
User story: As a Seller I want to manage user questions So that I build trust with buyers
Acceptance Criteria: Given: A buyer asks a question about one of my products. When: I receive a new question. Then: I am notified immediately of the new question. A response time of less than 24 hours is recommended.

4.3 Role: Database Administrator (DBA)

Title: Monitor Cluster Health Priority: High
User story: As a DBA I want to monitor cluster health So that the system remains highly available
Acceptance Criteria: Given: The database cluster is in operation. When: I monitor the cluster status. Then: I have a real-time dashboard to visualize the status. I receive automated alerts for any anomalies. Cluster logs are centralized for easy review.

Title: Implement Automated Backup Processes Priority: High
User story: As a DBA I want to implement automated backup processes So that critical data remains protected against loss or corruption
Acceptance Criteria: Given: The database contains critical information. When: The system executes backups. Then: Daily and weekly backups are performed. Backup restoration capability is verified. Backups are stored securely.

Title: Optimize Frequent Queries Priority: High
User story: As a DBA I want to optimize frequent queries So that response times remain low
Acceptance Criteria: Given: There are high-usage queries in the database. When: I perform optimizations on those queries. Then: Indexes and partitions are used to improve performance. Queries execute in under 1 second. Continuous performance monitoring of queries is in place.

Title: Configure Cross-Region Replication Priority: High
User story: As a DBA I want to configure cross-region replication So that there is fast access in multiple locations
Acceptance Criteria: Given: The database infrastructure extends to multiple regions. When: I configure replication. Then: Replicas are synchronized across regions. Latency between regions remains below 150 ms. Daily integrity validation of replicated data is performed.

Title: Control Access to Sensitive Data Priority: High
User story: As a DBA I want to control who can access sensitive data So that we prevent unauthorized use and meet security standards
Acceptance Criteria: Given: The database contains sensitive information. When: An attempt is made to access sensitive data. Then: Every access to sensitive data is audited. Active encryption is applied to sensitive data.

4.4 Role: Administrator

Title: Manage Product Categories Priority: High
User story: As a Administrator I want to manage product categories So that the platform stays organized
Acceptance Criteria: Given: I need to organize products on the platform. When: I manage categories. Then: I can create, edit, and delete categories. Changes to categories have an immediate impact on search filters.

Title: Access Real-Time Global Platform Metrics Priority: High
User story: As a Administrator I want to access real-time global platform metrics So that I can proactively identify and address performance issues
Acceptance Criteria: Given: I need to understand the overall platform performance. When: I review global metrics. Then: I can see reports by region, day, and category. Growth charts and alerts are displayed.

Title: Review and Moderate User-Generated Content Priority: High
User story: As a Administrator I want to review and moderate user-generated content So that the platform maintains high-quality listings and trustworthy reviews
Acceptance Criteria: Given: There is user-generated content (e.g., reviews, descriptions). When: I need to moderate this content. Then: I have a list of reported posts. I can perform actions such as approving, hiding, or deleting content. The responsible moderator for each action is logged.

Title: Monitor System Uptime Priority: High
User story: As a Administrator I want to monitor system uptime So that outages are prevented
Acceptance Criteria: Given: The system is in operation. When: I monitor the system status. Then: I receive alerts for latency or 500 errors. I have a real-time status panel for critical services. Error logs are centralized.

Title: Monitor System Uptime Priority: High
User story: As a Administrator I want to monitor system uptime So that outages are prevented
Acceptance Criteria: Given: The system is in operation. When: I monitor the system status. Then: I receive alerts for latency or 500 errors. I have a real-time status panel for critical services. Error logs are centralized.

4.5 Role: Platform Manager

Title: Analyze Sales Data Priority: High
User story: As a Manager I want to analyze sales data So that I can make strategic decisions
Acceptance Criteria: Given: I need to understand sales performance. When: I review sales data. Then: I have a consolidated dashboard. I see real-time KPIs. I can perform a monthly comparison.

Title: Evaluate Category Performance Priority: MEdium
User story: As a Manager I want to evaluate category performance So that I can prioritize promotions
Acceptance Criteria: Given: I need to identify which categories are performing best or worst. When: I analyze category performance. Then: I can view category rankings. Growth indicators are displayed. I receive automated BI suggestions for promotions.

Title: Compare Regional Metrics Priority: MEdium
User story: As a Manager I want to compare regional metrics So that I understand geographic trends
Acceptance Criteria: Given: The platform operates in multiple regions. When: I analyze regional metrics. Then: I can segment data by country/city. I can export reports in PDF format. Data is updated daily.

Title: Analyze Recommendation System Performance Metrics Priority: High
User story: As a Manager I want to analyze recommendation system performance metrics So that I can improve personalization and increase conversion rates
Acceptance Criteria: Given: The recommendation system is active. When: I review its performance. Then: I see a report of clicks vs. purchases per user. I can compare users with and without recommendations. I have visual dashboard indicators.

Title: Monitor Query Performance Metrics Priority: High
User story: As a Manager I want to monitor query performance metrics So that I can identify database bottlenecks and ensure optimal system responsiveness
Acceptance Criteria: Given: The database is in use. When: I review query performance. Then: I can see a list of the most used queries. Average execution times are displayed. I have a historical visualization of database load.

4.6 Role: BI analyst

Title: Study Sales by Region/Category Priority: High
User story: As a BI Analyst I want to study sales by region/category So that I can assess market impact
Acceptance Criteria: Given: I need to analyze platform sales. When: I access sales data. Then: I have an interactive dashboard with date filters. I can see aggregation of totals and trends. I can export data for presentations.

Title: Analyze Weekly Product Performance Trends Priority: Medium
User story: As a BI Analyst I want to analyze weekly product performance trends So that I can strategically plan promotions for high-potential and underperforming products
Acceptance Criteria: Given: I need to understand product performance. When: I analyze weekly performance trends. Then: I can see product rankings with key metrics. I can filter by brand, region, and seller. There is integration with the promotions module.

Title: Analyze Customer Engagement Patterns Priority: Medium
User story: As a BI Analyst I want to analyze customer engagement patterns So that i can develop targeted retention strategies cally plan promotions for high-potential and underperforming products
Acceptance Criteria: Given: I need to understand customer behavior. When: I analyze engagement patterns. Then: I see cart abandonment metrics. Average Browse time is displayed. I have a visualization of the conversion funnel.

Title: Extract Insights from Customer Reviews Priority: Medium
User story: As a BI Analyst I want to extract insights from customer reviews So that I can identify recurring product issues and improve quality control
Acceptance Criteria: Given: Customer reviews are available. When: I analyze the reviews. Then: Reviews are classified by negative keywords. Average rating graphs are available. I can apply filters by seller and category.

4.7 Role: Marketing Specialist

Title: Launch Segmented Campaigns Priority: High
User story: As a Marketer I want to launch segmented campaigns So that I maximize holiday sales
Acceptance Criteria: Given: I need to run marketing campaigns. When: I configure a campaign. Then: I can segment the campaign by region and category. I can define the duration of the campaign. I get an effectiveness report for the campaign.

Title: Monitor Real-Time Campaign Performance Metrics Priority: High
User story: As a Marketer I want to monitor campaign performance metrics in real-time So that I can optimize ad spend and maximize conversion rates
Acceptance Criteria: Given: A marketing campaign is active. When: I monitor its performance. Then: I have an analytics dashboard. I can apply campaign filters. I can export data for analysis.

Title: Prominently Feature Trending Products Priority: MEduium
User story: As a Marketer I want to prominently feature trending products on high-visibility pages So that I can increase site traffic and conversion opportunities
Acceptance Criteria: Given: Products are identified as trending by BI. When: I feature these products. Then: Product selection is automatic from BI. Products are placed on the homepage. I can evaluate the results of this action.

Title: Schedule and Automate Digital Ad Deployments Priority: High
User story: As a Marketer I want to schedule and automate digital ad deployments So that I can execute campaigns efficiently with perfect timing
Acceptance Criteria: Given: I need to launch digital ads. When: I configure ad deployment. Then: I have a campaign calendar for scheduling. An approval system is in place for ads.

5 Requirements Documentation

5.1 Functional Requirements

Table 1: Functional Requirements of the E-commerce Platform

ID	Requirement Name	Description	Required Inputs (Fields)
RF01	User Registration	The system must allow user registration with secure authentication.	Full Name, Email, ID Number, Address, Phone Number, Password
RF02	Product Search	The system must allow product searches using quick filters (category, price, rating).	Keyword, Category, Minimum Price, Maximum Price, Minimum Rating
RF03	Personalized Recommendations	The system must display recommendations based on user behavior.	User ID, Browsing History, Purchase History
RF04	Multiple Payment Methods	The system must allow payments via card, transfer, or cash on delivery.	Order ID, Payment Method, Card/Bank Info, Shipping Address
RF05	Publish Reviews	The system must allow reviews to be published only for purchased products.	User ID, Product ID, Rating, Comment
RF06	Create Listings	The system must allow products to be published with details and images.	Product Name, Description, Price, Stock, Images, Category
RF07	Product Statistics	The system must display statistics on views, sales, reviews, and stock.	Seller ID, Product ID, Date Range
RF08	Low Stock Alerts	The system must send notifications when stock is low.	Product ID, Threshold, Current Stock
RF09	Promotion Management	The system must allow creation of promotions with rules and duration.	Products, Discount, Start Date, End Date, Conditions
RF10	QA Management	The system must allow management of questions and answers in listings.	Product ID, Question ID, Response Text
RF11	Cluster Monitoring	The system must monitor node status and alerts.	System Logs, Node Metrics, Alert Thresholds
RF12	Multi-Region Replication	The system must allow data replication across regions with low latency.	Regions, Cluster Configuration
RF13	Automatic Backups	The system must schedule and store backups.	Date/Time, Backup Type, Destination

(Continued on next page)

ID	Requirement Name	Description	Required Inputs (Fields)
RF14	Query Optimization	The system must optimize response time for frequent queries.	Frequent Queries, Fields for Indexing/Partitioning
RF15	Category Management	The system must allow creation/modification of categories visible in filters.	Name, Parent Category, Description
RF16	Metrics by Region and Category	The system must display segmented key performance indicators.	Date, Region, Category, Metric Type
RF17	Content Moderation	The system must allow moderation of listings and reviews with traceability.	Content ID, Action, Justification, Moderator ID
RF18	System Error Notification	The system must alert the administrator in case of system errors.	Error Type, Affected Service, Date/Time, Severity
RF19	Sales KPIs Dashboard	The system must display key sales indicators in real time.	Date, Region, Category, KPI
RF20	Category/Region Performance	The system must visualize segment comparisons.	Date, Category, Region, Metric Type
RF21	Recommendation Effectiveness Report	The system must measure the effectiveness of personalized recommendations.	User ID, Recommended Products, Interactions, Purchases
RF22	Query Efficiency	The system must evaluate the performance and frequency of system queries.	Query Name, Execution Time, Frequency
End of table			

5.2 Non- functional Requirements

Table 2: Non-Functional Requirements of the E-commerce Platform

ID	Requirement Name	Description
NFR01	Big Data	The system must process and store large volumes of structured and unstructured data in both real-time and batch modes using centralized repositories for analysis.
NFR02	Fast Query Response	The system must ensure low latency for read/write operations using optimized transactional databases, in-memory caching, and advanced indexing for complex queries.
NFR03	Data Ingestion	The system must support massive batch ingestion and real-time streaming with fault tolerance and the ability to re-process data when needed.
NFR04	Business Intelligence	The system must integrate with analytics tools and provide real-time information for dashboards to strategic insights.
NFR05	Multi-Location Access	The system must provide global access through automatic geographic replication, proximity-based load balancing, and efficient delivery of static content to reduce latency.
NFR06	Recommendation System	The system must dynamically generate personalized recommendations, maintain user and product profiles, and update them in real-time based on recent interactions.
NFR07	High Availability	The system must ensure 24/7 operation using automatic data replication, disaster recovery mechanisms (failover), and continuous performance monitoring.
(Continued on next page)		

ID	Requirement Name	Description
NFR08	Scalability	The system must adapt to variable demand using horizontal scaling, data sharding, and elastic architecture with auto-scaling of resources.
NFR09	Security	The system must provide data encryption at rest and in transit, implement role-based access control (RBAC), and audit critical operations to prevent vulnerabilities.
NFR10	Regulatory Compliance	The system must comply with data privacy regulations and industry standards, including payment security certifications.
NFR11	Maintainability	The system must be modular to allow updates without interruption, include detailed technical documentation, and support automated deployment and monitoring.
NFR12	Cost Optimization	The system must optimize infrastructure usage through auto-scaling, cold storage for historical data, and strategies to avoid overprovisioning.
		End of table

6 ER Model

6.1 Components

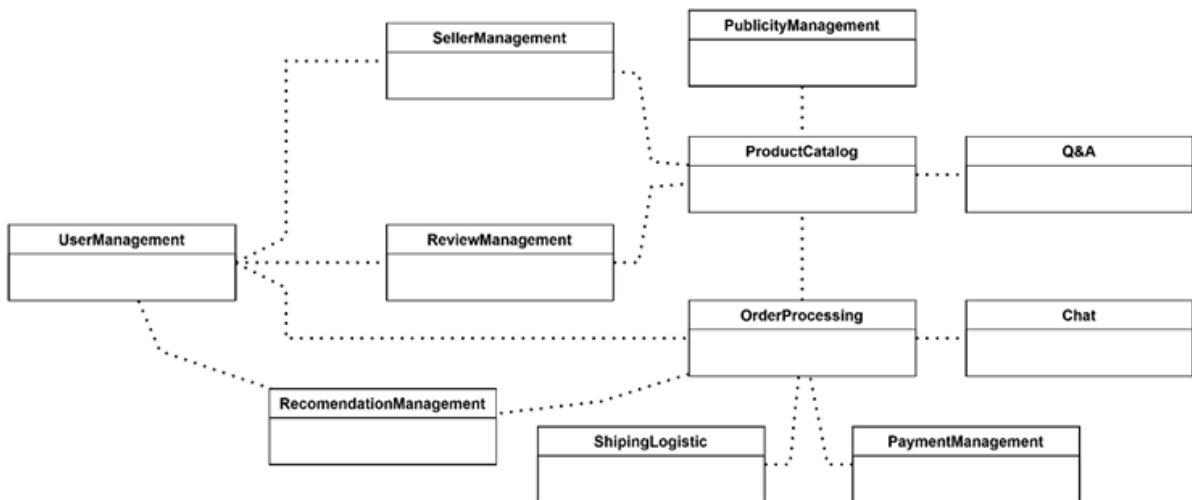


Figure 2: Components

6.2 Relationships

1. **UserManagement** has a relation with:
 - a. SellerManagement, if the user is a store owner.
 - b. OrdersProcessing
 - c. ReviewsManagement
2. **ProductCatalog** has a relation with:
 - a. SellerManagement
 - b. Orders
 - c. Reviews
 - d. Q&A, if some questions exist

3. **Orders** has a relation with:
 - a. PaymentManagement
 - b. ShippingLogistics
 - c. Chat (post-sale)
4. **RecommendationManagement**
 - a. UserManagement, to write all the queries from the customer
 - b. OrderProcessing, to get all the purchases from the customer
5. **PublicityManagement**
 - a. ProductCatalog

6.3 Entities

- **UserManagement**
 - E1. Users
 - E2. Roles
 - E3. Addresses
- **SellerManagement**
 - E4. Stores
- **ReviewManagement**
 - E5. Reviews
- **ProductCatalog**
 - E6. Products
 - E7. Categories
 - E8. ProductImages
 - E9. ProductDiscounts
- **Q&A**
 - E10. ProductQuestions
 - E11. ProductAnswers
- **OrderProcessing**
 - E12. Orders
 - E13. OrderDetails
 - E14. OrderStatuses
 - E15. Coupons
- **Chat**
 - E16. Conversations
 - E17. Messages
- **ShippingLogistic**
 - E18. Shipments
 - E19. ShipmentStatuses
- **PaymentManagement**

- E20. Payments
- E21. PaymentMethod
- E22. PaymentStatuses
- **RecommendationManagement**
 - E23. UserSearchHistory
- **PublicityManagement**
 - E24. Campaign
 - E25. Ad

6.4 Attributes per Entity

Table 3: Entities of the E-commerce Platform

ID	Entity Name	Attributes
E1	Users	id, name, email, password, phone, birthday, createdAt
E2	Roles	id, nameRole
E3	Addresses	id, address, city, zipCode, country
E4	Stores	id, name, isOfficial, createdAt
E5	Reviews	id, rating, coment, createdAt
E6	Products	id, title, description, price, stock, createdAt
E7	Categories	id, nameCategories
E8	ProductImages	id, imageURL
E9	ProductDiscounts	id, percentage, startDate, endDate, isActive
E10	ProductQuestions	id, questionText, createdAt
E11	ProductAnswers	id, answerText, createdAt
E12	Orders	id, totalPrice, createdAt
E13	OrderDetails	id, quantity, unitPrice
E14	OrderStatuses	id, nameOrderStatus
E15	Coupons	id, code, isPercentage, valuePercentage, valueFixed, isFixed, maxUsage, usageCount, expirationDate, isActive
E16	Conversations	id, createdAt
E17	Messages	id, textMessage, isRead, sendAt
E18	Shipments	id, trackingNumber, carrier, shippedAt, deliveredAt
E19	ShipmentStatuses	id, nameShipmentStatus
E20	Payments	id, amount, createdAt
E21	PaymentMethod	id, methodName
E22	PaymentStatuses	id, namePaymentStatus
E23	UserSearchHistory	id, searchQuery, date
E24	Campaign	id, name, startDate, endDate, status, targetKeywords
E25	Ad	id, imageURL, bidAmount, clicksAmount

End of table

6.5 Relationship Matrix

E	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
1		X	X	X						X		X					X						X		
2	X																								
3	X																								
4	X				X	X					X													X	
5				X																					
6				X			X	X	X	X			X												
7					X																				
8					X																				
9					X																				
10	X				X																				
11			X																						
12	X												X	X	X	X		X		X					
13						X						X													
14												X													
15												X													
16												X					X								
17	X																X								
18												X							X						
19																		X							
20												X								X	X				
21																			X						
22																			X						
23	X																								
24			X																					X	
25																								X	

Figure 3: Relationship Matrix

6.6 Relation Types

Table 4: Relation Types of the E-commerce Platform

From Entity	To Entity	Description
E1.Users 1..1	E2.Roles 1..N	Each user has one role (mandatory). A role can be assigned to multiple users.
E1.Users 0..N	E3.Addresses 1..1	A user can have 0 or many addresses. Each address belongs to a single user.
E1.Users 0..1	E4.Stores 1..1	A user can own 0 or one store. Each store belongs to a single user.
E1.Users 0..N	E10.ProductQuestions 1..1	A user can have 0 or many questions. Each question belongs to one user.

(Continued on next page)

From Entity	To Entity	Description
E1.Users 0..N	E12.Orders 1..1	A user can have 0 or many orders. Each order belongs to a single user.
E1.Users 0..N	E5.Reviews 1..1	A user can have 0 or many reviews. Each review belongs to a single user.
E1.Users 0..N	E23.UserSearchHistory 1..1	A user can have 0 or many searches. Each search belongs to one user.
E1.Users 0..N	E17.Messages 1..1	A user can have 0 or many messages. Each message belongs to one user.
E4.Stores 0..N	E6.Products 1..1	A store can have 0 or many products. Each product belongs to one store.
E4.Stores 0..N	E11.ProductAnswer 1..1	A store can have 0 or many answers. Each answer belongs to one store.
E4.Stores 0..N	E24.Campaign 1..1	A store can have 0 or many campaigns. Each campaign belongs to one store.
E6.Products 1..1	E7.Categories 1..N	Each product belongs to one category (mandatory). A category can have many products.
E6.Products 1..N	E8.ProductImages 1..1	Each product must have at least 1 image. Each image belongs to one product.
E6.Products 0..N	E9.ProductDiscounts 1..1	A product can have 0 or many discounts. Each discount belongs to one product.
E6.Products 0..N	E10.ProductQuestions 1..1	A product can have 0 or many questions. Each question belongs to one product.
E6.Products 0..N	E5.Reviews 1..1	A product can have 0 or many reviews. Each review belongs to one product.
E10.ProductQuestions 0..N	E11.ProductAnswer 1..1	A question can have 0 or many answers. Each answer belongs to one question.
E12.Orders 1..N	E13.OrderDetails 1..1	Each order must contain at least 1 product. Each detail belongs to one order.
E12.Orders 1..1	E14.OrderStatuses 1..N	Each order has one status (mandatory). A status can apply to many orders.
E12.Orders 0..1	E15.Coupons 0..N	An order may use 0 or 1 coupon. A coupon can be used in 0 or many orders.
E12.Orders 0..N	E16.Conversations 1..1	An order may use 0 or many conversations (per seller). Each conversation belongs to one order.
E12.Orders 1..1	E18.Shipments 1..1	Each order has one shipment (mandatory). Each shipment belongs to one order.
E12.Orders 1..1	E20.Payments 1..1	Each order has one payment (mandatory). Each payment belongs to one order.
E13.OrderDetails 1..1	E6.Products 0..N	Each order detail belongs to one product. A product can appear in 0 or many orders.
E16.Conversations 1..N	E17.Messages 1..1	Each conversation must have at least 1 message. Each message belongs to one conversation.
E18.Shipments 1..1	E19.ShipmentStatuses 1..N	Each shipment has one status (mandatory). A status can apply to many shipments.
E20.Payments 1..1	E21.PaymentMethod 1..N	Each payment uses one method (mandatory). A method can be used in many payments.
E20.Payments 1..1	E22.PaymentStatuses 1..N	Each payment has one status (mandatory). A status can apply to many payments.
E24.Campaign 1..N	E25.Ad 1..1	Each campaign must contain at least 1 ad. Each ad belongs to one campaign.
End of table		

6.7 First Model

First Model only with entities and relationships

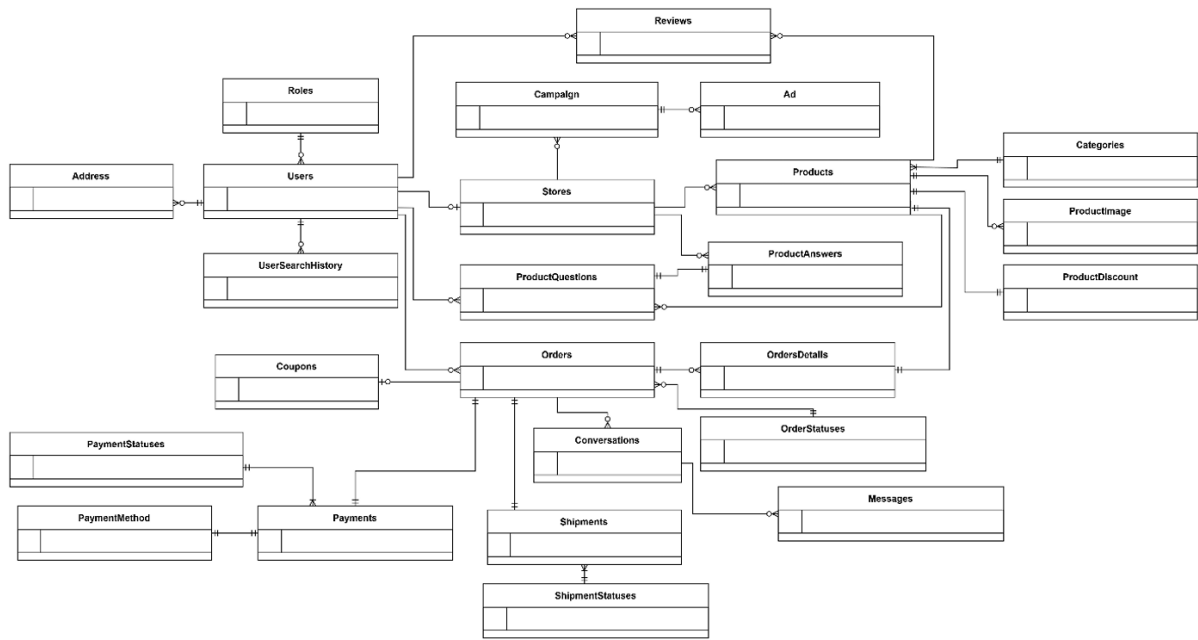


Figure 4: First model ER

6.8 Second Model

Second Model with attributes and data types

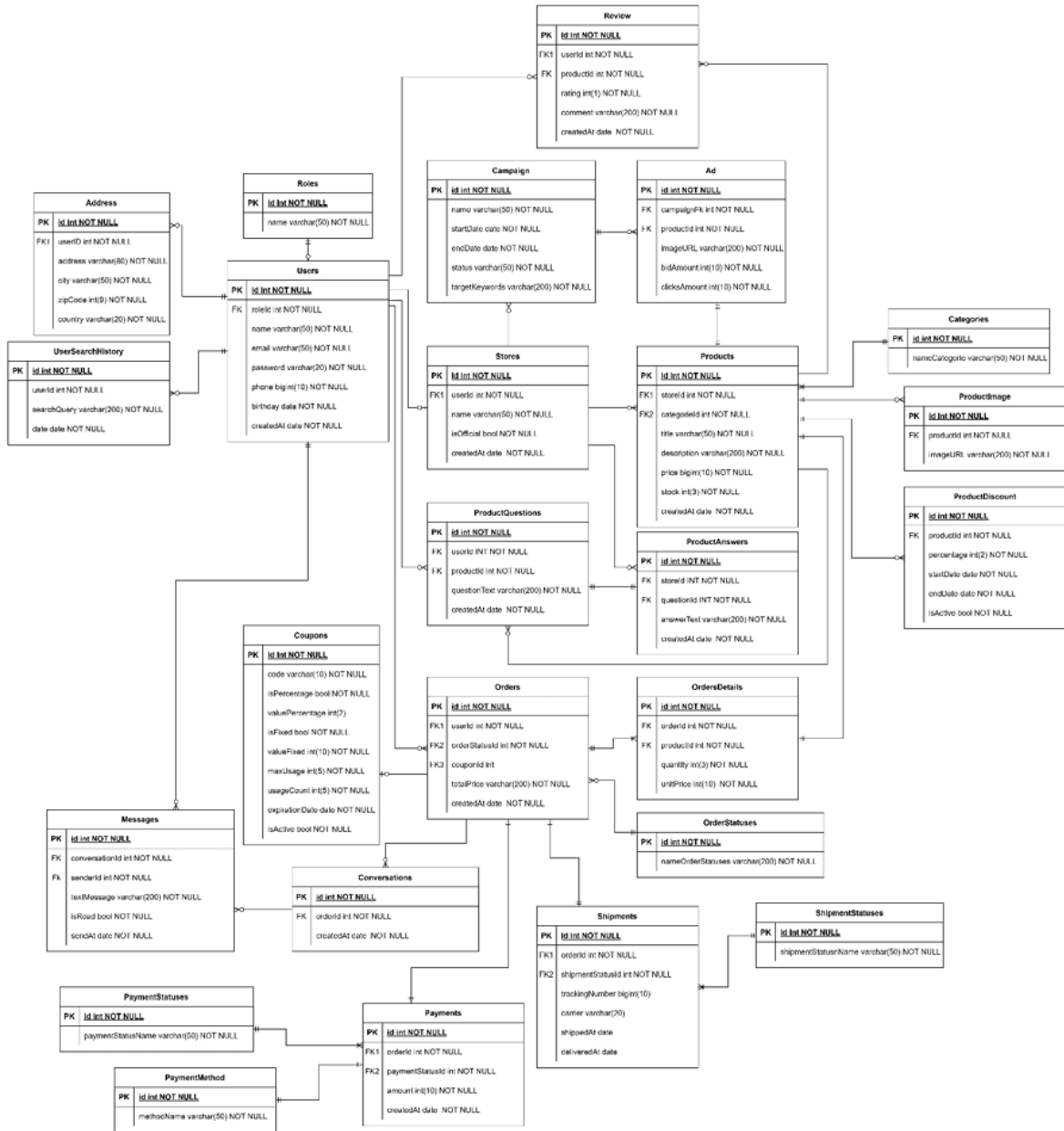


Figure 5: Second model ER

7 Database Architecture

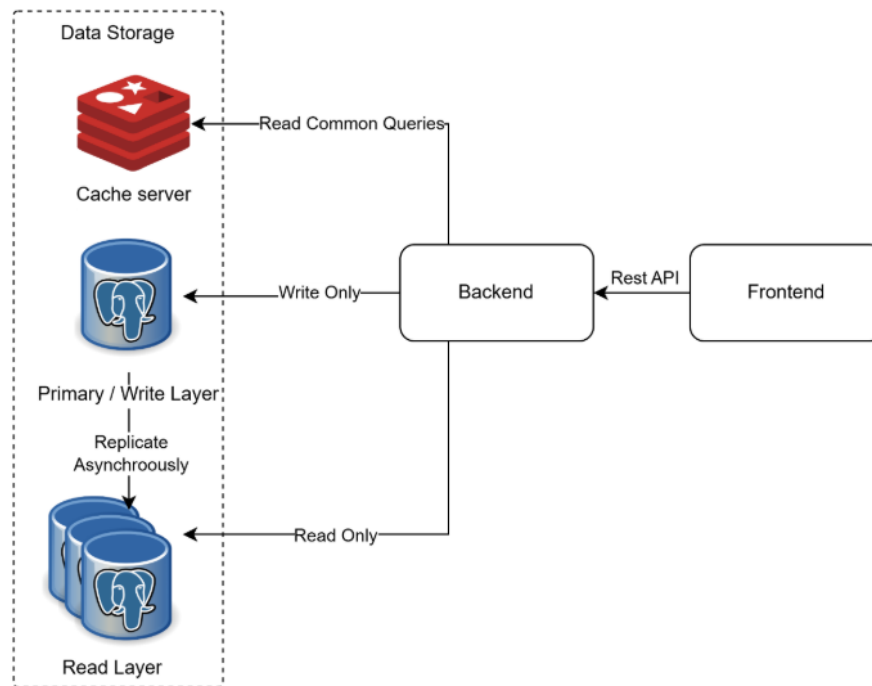


Figure 6: Database architecture

Main Components

1. Write Layer

Technology: PostgreSQL.

Responsibility: Write operations like INSERT, UPDATE, DELETE.

2. Read Layer

Technology: PostgreSQL.

Responsibility: Read operations like SELECT, and horizontally scalable replicas under load.

3. Cache Layer

Technology: Redis.

Use pattern: The application searches first in Redis. If the data doesn't exist, it queries the database. All writes to the primary database are also written to Redis.

Data in cache: • Popular products: saving titles, prices, and discounts.

- Daily offers: saving titles, prices, and discounts.
- User data (tokens).

TTL: • Popular products: 24 hours.

- Daily offers: 24 hours.
- User data: 30 minutes.

8 Data System Architecture

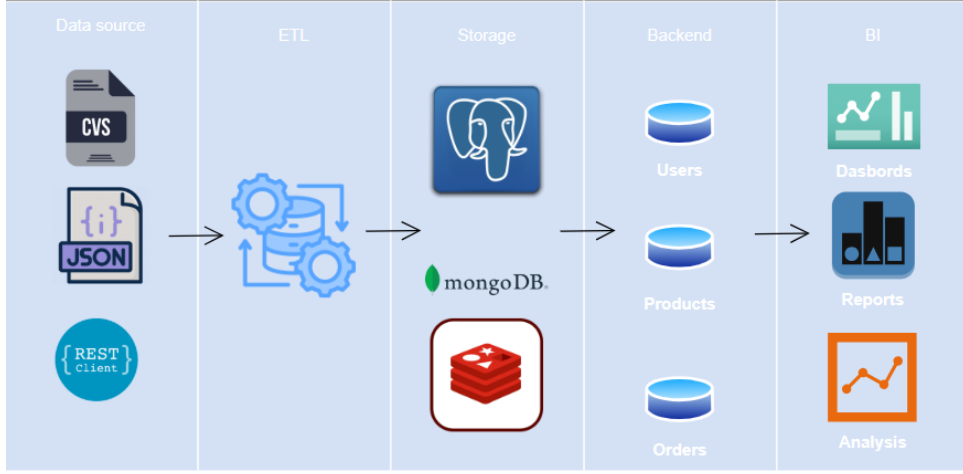


Figure 7: Data system architecture

The proposed system is designed for an e-commerce platform like Mercado Libre, where the high volume of transactions, user interactions, and analytical requirements demand a scalable and efficient architecture. The key components, their functions, and the justification for the selected technologies are described below, based on distributed systems architectural principles and real-world platform experiences.

8.1 Components and tools

- API Gateway and Backend Services:

Since the project does not include frontend development, the API Gateway and Backend Services act as the primary access layer for any external systems or clients (such as testing tools, REST clients, or future applications). The API Gateway manages and routes incoming HTTP requests, handling responsibilities such as authentication, registration, rate limiting, and request validation. This abstraction decouples external consumers from the system's internal structure, providing a stable and secure entry point. Behind the gateway, the backend services implement the system's core logic in a modular fashion. These services manage operations such as user registration, product and order management, inventory control, and data aggregation. Designed following the principles of service-oriented architecture (SOA) or microservices, they promote scalability, maintainability, and fault isolation.

These services interact directly with the data storage components (PostgreSQL, MongoDB, and Redis), acting as a centralized data access layer and ensuring consistency and security across all data operations.

- Transactional Database Layer (PostgreSQL):

For each transactional operation, such as registering users, processing orders, or updating inventory, the system uses PostgreSQL as the primary relational database engine. This technology was selected for its strong adherence to ACID (Atomicity, Consistency, Isolation, and Durability) properties, ensuring integrity and reliability in high-volume and high concurrency scenarios. Additionally, PostgreSQL offers advanced capabilities such as native support for semi-structured structures (e.g., JSON), extensibility through user-defined functions, and horizontal read scaling options using replicas. These features make it a highly robust and adaptable solution for managing structured data in mission-critical transaction-oriented systems.

- Cache Layer (Redis):

To optimize the performance of read-intensive operations, the architecture incorporates Redis as an in-memory cache layer. Redis is a high-performance key-value store that operates entirely in RAM, enabling sub millisecond response times for frequently accessed data. This makes it particularly suitable for scenarios where low latency and high throughput are crucial, such as generating dynamic product listings, managing active user sessions, or delivering real-time promotional content.

By offloading repeated read operations from the main database, Redis helps improve scalability and reduce the load on the transactional system. Its support for configurable expiration policies (TTL, time-to-live), persistence options, and atomic operations aligns with best practices for modern distributed system design.

- NoSQL Database (MongoDB):

To efficiently manage semi-structured and constantly evolving data, the system incorporates MongoDB, a document oriented NoSQL database designed for flexibility and scalability. In the context of an e-commerce platform, MongoDB is particularly well-suited for storing user generated content such as product reviews, search histories, Q&A interactions, and marketing campaign metadata. Its schemeless model allows the database to dynamically adapt to variations in data structure without requiring costly migrations or rigid constraints, which is essential for capturing real-time behavioral data in a variety of formats. Additionally, MongoDB supports horizontal scaling through sharding, enabling it to handle large volumes of non uniform and write-intensive data, typical of user interaction logs. These capabilities align with architectural best practices for systems that require high agility in content management and extracting insights from user behavior patterns.

- ETL Pipeline:

Since the project relies on realistic but simulated datasets, the ETL (Extract, Transform, Load) process is specifically designed to ingest structured files (e.g., CSV, JSON) representing key business entities such as orders, users, products, reviews, and marketing campaigns. These datasets are either manually curated or synthetically generated to reflect the behavior and structure of a real e-commerce platform. This setup allows for controlled testing of the data system architecture without relying on external sources in real time.

In the extraction phase, the system periodically ingests data from local or cloud repositories (e.g., GitHub datasets, S3 buckets, or mounted volumes). Each dataset is categorized (e.g., orders.csv, products.json, reviews.csv) and versioned to simulate data updates over time. For automation, batch processing tools such as Apache Airflow are used to trigger ingestion workflows at scheduled intervals. Alternatively, Apache NiFi can be used for flexible, visual control of file-based flow ingestion.

During the transformation phase, raw datasets undergo data cleansing, type normalization, enrichment (e.g., total order value calculation, review sentiment classification), and structural adaptation to fit relational (PostgreSQL) and NoSQL (MongoDB) models. For example, denormalized product data is split into atomic tables (products, categories, stock) for PostgreSQL, while user behavior logs are reformatted into JSON documents for MongoDB. In this case, Python (with Pandas) or SQL scripts are used for the transformation logic due to their flexibility and simplicity for small and medium data volumes.

Finally, in the loading phase, the transformed data is inserted into the corresponding systems: the main business data (orders, users, payments) in PostgreSQL, and the analytical or behavioral data (search history, reviews, Q&A threads) in MongoDB. This hybrid loading ensures transactional consistency and analytical preparation. The tailored ETL pipeline is essential for simulating realistic data flows and testing the architecture's ability to handle various data types. It supports incremental loads and repeatable testing, crucial in academic or prototyping environments. Furthermore, this simulated setup maintains the architectural integrity of a real-production e-commerce system.

- Business Intelligence & Dashboards (Metabase or Power BI):

For business data analysis and visualization, the architecture integrates a Business Intelligence (BI) layer with open-source tools that ensure simple implementation and no licensing costs. In this context, Metabase stands out as a highly effective solution thanks to its ease of implementation, intuitive interface, and native connectivity with both relational databases such as PostgreSQL and NoSQL systems, in this case MongoDB. Through simple configuration, Metabase enables the creation of interactive dashboards, KPI visualizations, and ad hoc queries accessible to a variety of stakeholder roles, including platform administrators, marketing analysts, and business intelligence professionals. Its web-based interface facilitates collaborative data exploration without advanced technical knowledge, making it easy to integrate into agile development environments. This makes it particularly suitable for academic or simulated e-commerce platforms, where rapid insights and low operational costs are essential. Additionally, its support for scheduled reports and data alerts facilitates decision-making through proactive monitoring.

- Cloud Infrastructure (AWS RDS, MongoDB Atlas)

These components are interconnected to enable a continuous flow of operational and analytical data. Data from user actions such as browsing and purchases is processed by backend services, stored in the corresponding databases, and periodically transformed and analyzed using ETL and BI tools.

8.2 Data flow

- Flow between the API Gateway, Backend Services, and Databases

The data flow begins at the API Gateway and Backend Services layer, which acts as the central entry point for all external requests. When an external client (such as a testing tool or automated system) makes a request—for example, to register a user or query a product—the API Gateway routes this request to the appropriate backend service. Once the request is validated, the backend directly accesses the storage systems: PostgreSQL for structured and transactional data (such as users, orders, and products), and MongoDB for unstructured or dynamic data (such as searches, reviews, or behavior logs). For frequent read operations, such as retrieving featured products, the backend also queries Redis, which stores data in memory for faster response times. This flow ensures a clear separation between business logic, data access, and service exposure.

- Flow between Backend, PostgreSQL, and Redis

When the backend needs to perform critical operations such as entering an order or updating inventory, it writes directly to PostgreSQL, ensuring consistency through its ACID properties. At the same time, to improve performance in subsequent queries, the Redis layer can be updated with newly processed data (for example, new product stock). In scenarios where rapid access to frequently queried data is required, such as lists of best-selling products, the backend first checks Redis. If the information is not found there or is outdated, it queries PostgreSQL, retrieves the data, and stores it back in Redis with a configured expiration policy. This flow optimizes response times without compromising the integrity of the underlying data.

- Flow between Backend and MongoDB

The backend also interacts with MongoDB to store and query user-generated data that doesn't require a rigid structure. For example, when a new product review or user inquiry (Q&A) is received, the backend structures the content as a JSON document and saves it to MongoDB. This data can also be queried by analysts or other services that need to study customer behavior, search patterns, or campaign interactions. This flow leverages MongoDB's flexibility to store heterogeneous and constantly evolving information without affecting the core relational model.

- ETL Process Flow

The data flow in the ETL process begins with the extraction of structured files (CSV or JSON) containing synthetic information, such as products, users, orders, and campaigns. These files are extracted from local repositories or simulated cloud sources. In the transformation phase, the data is cleaned, normalized, and adapted: entities are separated, formats are converted, and derived fields are generated (for example, the total value of a purchase or the rating of a review by category). Subsequently, in the loading phase, the data is inserted into its respective destinations: PostgreSQL for transactional information and MongoDB for behavioral data or user-generated content. This flow allows the architecture to remain functional and representative of the real world without relying on live data sources.

- Flow to the BI Module

Once the data has been stored in PostgreSQL and MongoDB, the Business Intelligence (BI) system accesses these sources to generate reports, dashboards, and interactive visualizations. Metabase connects directly to both databases to view metrics such as sales by category, review trends, most visited products, or search trends. This data is presented to users such as administrators or analysts, who can explore it without advanced technical knowledge. The flow of data from the databases to Metabase can be scheduled periodically or triggered on demand, allowing for both real-time visualizations and historical reports.

- Cloud Infrastructure Flow

While the system is designed for an academic environment, its implementation in cloud infrastructure is planned using services such as AWS RDS (for PostgreSQL) and MongoDB Atlas (for Mon-

goDB). In this context, the data flow follows the same principles but benefits from additional capabilities such as automatic replication, geographic load balancing, and continuous backup. These platforms allow backend services, the ETL process, and the BI tool to interact with the databases in a transparent, scalable, and secure manner. This flow ensures operational continuity and simulates the real-world availability and scalability conditions of commercial platforms.

9 Information Requirements

An e-commerce system must effectively manage information to ensure smooth operations, enhance user experience, and optimize business strategies. Below are the main data types the system should handle and analyze.

9.1 Sales & Financial Reporting

- **Gross Merchandise Volume (GMV) by Seller**

- *Data Source:* PostgreSQL (Transactional)
- *Schema Source:* `stores`, `orderDetails`
- *Business Link:* Platform Manager (p.5)
- *Desired Output:*
$$\begin{bmatrix} \text{Seller} & \text{Total Sales} \\ \text{Store A} & \$25,400 \\ \vdots & \vdots \end{bmatrix}$$

- **Coupon Effectiveness**

- *Data Source:* PostgreSQL (Transactional)
- *Schema Source:* `coupons`, `orders`
- *Business Link:* RF09 (Promotions)
- *Desired Output:*
 - * Redemption rate: 68%
 - * Revenue lift: +\$12,400

9.2 User Behavior Analysis

- **Search-to-Purchase Conversion**

- *Data Source:* MongoDB (Analytical)
- *Schema Source:* Aggregated from `usersearchhistory` and `orderDetails`
- *Business Link:* BI Analyst (p.6)
- *Desired Output:*
 1. Searches: 10,240
 2. Purchases: 310 (3.0%)

- **Review Sentiment Analysis**

- *Data Source:* MongoDB (Analytical)
- *Schema Source:* Processed from `reviews.comment`
- *Business Link:* RF26
- *Desired Output:*
 - * Positive: 68%
 - * Negative: 12%
 - * Neutral: 20%

9.3 Inventory & Catalog Management

- **Stock-Out Risk**

- *Data Source:* PostgreSQL (Transactional)
- *Schema Source:* `products.stock`, `orderDetails`
- *Business Link:* Seller (p.4)
- *Desired Output:*
 - * Product ID 7821: 12 units left
 - * Product ID 4015: 8 units left

- **Category Trends**

- *Data Source:* MongoDB (Analytical)
- *Schema Source:* Aggregated from `categories`, `orderDetails`
- *Business Link:* Admin (p.5)
- *Desired Output:*
 - * Electronics: +15% MoM
 - * Fashion: -3% MoM

9.4 Marketing & Campaign Analytics

- **Campaign ROI**

- *Data Source:* MongoDB (Analytical)
- *Schema Source:* Aggregated from `ad`, `orders`
- *Business Link:* Marketing (p.7)
- *Desired Output:*
 - * Campaign A: \$4.20 return per \$1

- **Keyword Accuracy**

- *Data Source:* MongoDB (Analytical)
- *Schema Source:* `campaign.target.Keywords` vs `usersearchhistory`
- *Business Link:* RF27
- *Desired Output:*
 - * "Gaming chair": 91% match rate

9.5 Order Fulfillment & Logistics

- **Delivery Timeline**

- *Data Source:* PostgreSQL (Transactional)
- *Schema Source:* `shipments.shipped_At`, `delivered_At`
- *Business Link:* Customer
- *Desired Output:*
 - * Avg. delivery: 38.2 hours

- **Carrier Performance**

- *Data Source:* MongoDB (Analytical)
- *Schema Source:* Aggregated from `shipments`
- *Business Link:* Logistics Partners
- *Desired Output:*
 - * Carrier X: 98% on-time

10 Query Proposal

At this stage of the project, synthetic data has been generated with the aim of evaluating the correct formulation and functionality of the proposed queries on the database. Although a detailed performance analysis is not conducted at this point, the data generation is intended to prepare the necessary environment for more rigorous performance testing in future stages of the project. For now, the focus is on verifying that the queries produce the expected results and meet the previously defined functional requirements.

10.1 Sales & Financial Reporting

10.1.1 Gross Merchandise Volume (GMV) by Seller

```
1 SELECT
2   s.name AS store,
3   SUM(od.unit_price * od.quantity) AS total_sales
4 FROM sellermanagement.stores s
5 JOIN productcatalog.products p ON s.id = p.store_id
6 JOIN orderprocessing.orderdetails od ON p.id = od.product_id
7 GROUP BY s.id;
```

Listing 1: PostgreSQL

Purpose: Calculates total sales per store for commission settlements.

Insight: Identifies top-performing sellers and revenue distribution.

	store character varying (50)	total_sales bigint
1	ÚnicoFlea1	67188632
2	PuraBazaar2	50518168
3	MajesticGalería3	50290367
4	RoyalLaden4	58990329
5	LuckyKaufhaus5	61003786
6	LuxeBoutique6	65941726
7	VivaComercio7	53719706
8	HappyShop8	58911710
9	FrescoBazaar9	51895739
10	RapidDepot10	60094762
11	SuperDepot11	66002406
Total rows: 1000 of 100000		Query complete 00:00:07.379

Figure 8: Listing 1 Result

10.1.2 Coupon Effectiveness

```
1 SELECT
2   c.code,
3   COUNT(o.id) AS redemption_count,
4   SUM(o.total_price) AS revenue_impact
5 FROM orderprocessing.coupons c
6 JOIN orderprocessing.orders o ON c.id = o.coupon_id
7 WHERE c.is_active = true
8 GROUP BY c.id;
```

Listing 2: PostgreSQL

Purpose: Measures coupon usage and revenue impact.

Insight: Shows ROI of discount campaigns and popular promotions.

10.1.3 User Behavior Analysis

10.2 Search-to-Purchase Conversion

	code character varying (10)	redemption_count bigint	revenue_impact numeric
9	GAMER10	103241	174659071618
10	PETLOVE	103669	175502138039
11	READ20	103010	174607508591
12	TECH25	103407	174571673037
13	BEAUTY5	103128	174909606984
14	FALLSALE	102993	173950003610
15	WELCOME50	102807	173808263443
16	FLASH25	103724	175475366541
17	JULY4TH	103483	174480031074
18	VIP30	103363	174917038745
Total rows: 18 of 18		Query complete 00:00:00.586	

Figure 9: Listing 2 Result

```

1 db.user_behavior.aggregate([
2   {
3     $group: {
4       _id: "$search_query",
5       searches: { $sum: 1 },
6       purchases: { $sum: { $cond: [{ $gt: ["$order_id", null] }, 1, 0] } }
7     }
8   },
9   {
10    $project: {
11      conversion_rate: { $round: [{ $multiply: [
12        { $divide: ["$purchases", "$searches"] }, 100] }, 2] }
13    }
14  }
15 ]);

```

Listing 3: MongoDB Aggregation

Purpose: Analyzes conversion rates from search terms to purchases.

Insight: Identifies high-intent keywords and content gaps.

10.3 User Behavior Analysis

10.3.1 Search-to-Purchase Conversion

```

1 db.reviews.aggregate([
2   {
3     $project: {
4       positive: { $cond: [{ $gte: ["$rating", 4] }, 1, 0] },
5       negative: { $cond: [{ $lte: ["$rating", 2] }, 1, 0] }
6     }
7   },
8   {
9     $group: {
10      _id: "$product_id",
11      positive_pct: { $avg: "$positive" },
12      negative_pct: { $avg: "$negative" }
13    }
14  }
15 ]);

```

Listing 4: MongoDB Aggregation

Purpose: Quantifies review sentiment per product.

Insight: Flags products needing quality intervention.

10.3.2 Review Sentiment Analysis

```

1 db.reviews.aggregate([
2   {
3     $project: {

```

```

4     positive: { $cond: [{ $gte: ["$rating", 4] }, 1, 0] },
5     negative: { $cond: [{ $lte: ["$rating", 2] }, 1, 0] }
6   }
7 },
8 {
9   $group: {
10    _id: "$product_id",
11    positive_pct: { $avg: "$positive" },
12    negative_pct: { $avg: "$negative" }
13  }
14 }
15 ]);

```

Listing 5: MongoDB Aggregation

Purpose: Quantifies review sentiment per product.

Insight: Flags products needing quality intervention.

10.4 Inventory & Catalog Management

10.4.1 Stock-Out Risk

```

1 SELECT
2   p.id,
3   p.title,
4   p.stock,
5   (SELECT AVG(quantity)
6    FROM orderprocessing.orderdetails
7    WHERE product_id = p.id
8      AND created_at > CURRENT_DATE - INTERVAL '30 days'
9   ) AS avg_daily_demand
10 FROM productcatalog.products p
11 WHERE p.stock < (SELECT AVG(quantity) * 2
12                  FROM orderprocessing.orderdetails
13                  WHERE product_id = p.id);

```

Listing 6: PostgreSQL

Purpose: Identifies products at risk of stock depletion.

Insight: Triggers restocking alerts when inventory $\leq 2 \times$ daily demand.

	id [PK] integer	title character varying (50)	stock integer	avg_daily_demand numeric
1	1	Portable Speaker	20	[null]
2	3	Compact Shoes	5	[null]
3	4	Mega Jacket	1	[null]
4	5	Durable Tablet	3	[null]
5	7	Lightweight Keyboard	23	6.333333333333333
6	8	Powerful Backpack	7	4.750000000000000
7	9	High-Tech Camera	10	[null]
8	10	Wireless Watch	15	[null]

Figure 10: Listing 6 Result

10.4.2 Category Trends

```

1 db.category_performance.aggregate([
2   {
3     $group: {
4       _id: "$category_id",
5       current_month: { $sum: "$sales" },
6       previous_month: { $sum: "$previous_sales" }
7     }
8   },
9   {
10    $project: {
11      growth_pct: {
12        $multiply: [
13          { $divide: [

```

```

14         { $subtract: [ "$current_month", "$previous_month" ] },
15         "$previous_month"
16     ]},
17     100
18 ]
19 }
20 }
21 }
22 ]);

```

Listing 7: MongoDB Aggregation

Purpose: Tracks MoM category sales growth.

Insight: Guides merchandising decisions and category investments.

10.5 Marketing & Campaign Analytics

10.5.1 Campaign ROI

```

1 db.campaigns.aggregate([
2   {
3     $lookup: {
4       from: "ads",
5       localField: "_id",
6       foreignField: "campaign_id",
7       as: "ads"
8     }
9   },
10  {
11    $unwind: "$ads"
12  },
13  {
14    $project: {
15      cost: "$ads.bid_amount",
16      revenue: { $multiply: [
17        "$ads.click_amount",
18        { $divide: ["$ads.conversion_revenue", 1000] }
19      ]}
20    }
21  },
22  {
23    $group: {
24      _id: "$_id",
25      roi: { $sum: { $divide: ["$revenue", "$cost"] } }
26    }
27  }
28 ]);

```

Listing 8: MongoDB Aggregation

Purpose: Calculates return-on-investment per campaign.

Insight: Optimizes ad spend allocation to high-performing campaigns.

10.5.2 Keyword Accuracy

```

1 db.campaigns.aggregate([
2   {
3     $project: {
4       keywords: { $split: ["$target_keywords", ","] },
5       campaign_id: 1
6     }
7   },
8   {
9     $unwind: "$keywords"
10  },
11  {
12    $lookup: {
13      from: "search_logs",
14      localField: "keywords",
15      foreignField: "query",
16      as: "matches"

```

```

17 }
18 },
19 {
20   $project: {
21     match_count: { $size: "$matches" }
22   }
23 },
24 {
25   $group: {
26     _id: "$campaign_id",
27     match_rate: { $avg: "$match_count" }
28   }
29 }
30 });

```

Listing 9: MongoDB Aggregation

Purpose: Measures keyword-targeting alignment.

Insight: Improves relevance of future campaign targeting.

10.6 Order Fulfillment & Logistics

10.6.1 Delivery Timeline

```

1 SELECT
2   EXTRACT(EPOCH FROM (delivered_at - shipped_at))/3600 AS delivery_hours,
3   COUNT(*) AS shipments
4 FROM shippinglogistic.shipments
5 WHERE delivered_at IS NOT NULL
6 GROUP BY 1;

```

Listing 10: PostgreSQL

Purpose: Calculates actual delivery time distributions.

Insight: Identifies logistics bottlenecks and SLA compliance.

	delivery_time_hours numeric	shipments bigint
1	24.0000000000000000	85208
2	192.0000000000000000	143090
3	240.0000000000000000	142807
4	168.0000000000000000	142966
5	216.0000000000000000	142634
6	72.0000000000000000	229676
7	48.0000000000000000	227254
8	96.0000000000000000	228174
9	144.0000000000000000	142618
10	120.0000000000000000	228507

Figure 11: Listing 10 Result

10.6.2 Carrier Performance

```

1 db.shipments.aggregate([
2   {
3     $match: { status: "Delivered" }
4   },
5   {
6     $group: {
7       _id: "$carrier",

```



```

8     avg_delivery_hrs: { $avg: "$delivery_hours" },
9     on_time_rate: {
10         $avg: {
11             $cond: [{ $lte: ["$delivery_hours", 48] }, 1, 0]
12         }
13     }
14 }
15 }
16 ]);

```

Listing 11: MongoDB Aggregation

Purpose: Benchmarks carrier delivery reliability.

Insight: Informs carrier selection and contract negotiations.

10.7 Redis Caching in Spring Boot

10.7.1 Personalized Recommendations

```

1 @Service
2 public class RecommendationService {
3
4     @Cacheable(value = "userRecs", key = "#userId",
5               unless = "#result == null || #result.isEmpty()")
6     public List<Product> getRecommendations(String userId) {
7         return behaviorRepo.findTopProducts(userId);
8     }
9 }

```

Listing 12: Spring Boot Service

Purpose: Caches personalized product recommendations.

Benefit: Reduces latency from 2s → 15ms for recommendation API calls.

10.7.2 Product Detail Caching

```

1 @Service
2 public class ProductService {
3
4     @Cacheable(value = "productDetails", key = "#productId",
5               condition = "#productId > 100",
6               unless = "#result.stock < 10")
7     public Product getProduct(Long productId) {
8         return productRepo.findWithDiscount(productId);
9     }
10
11     @CacheEvict(value = "productDetails", key = "#product.id")
12     public void updateProduct(Product product) {
13         productRepo.save(product);
14     }
15 }

```

Listing 13: Spring Boot Service

Purpose: Caches product details with stock-aware eviction.

Benefit: Improves product API performance during traffic spikes.

10.8 Technology Integration Strategy

- **PostgreSQL:** All transactional operations (orders, inventory, payments)
- **MongoDB:** Behavioral analytics and historical trends
- **Redis:** High-velocity read optimization (caching)
- **ETL Pipeline:** Nightly sync from PostgreSQL → MongoDB

A Appendix

A.1 Feedback

User stories are changed to a card format for easier viewing.